



Task Parallel Library (TPL)

TPL

- Vereinfacht die Parallelisierung
- Ersetzt/Vereinfacht Multithreading
- Nutzt alle zur Verfügung stehenden Prozessorkerne
- Teilt die Arbeit automatisch auf mehrere Kerne auf und kümmert sich um Low-Level Details
- Klassen:
 - Task<T>
 - ValueTask<T>

Tasks erstellen

- Tasks können erstellt und konfiguriert werden
- Tasks können auch ohne weitere Konfiguration gestartet werden

```
Task t1 = new Task(MachWas);  
t1.Start();  
  
//.NET 4.0  
Task t2 = Task.Factory.StartNew(MachWas);  
//.NET 4.5  
Task t3 = Task.Run(new Action(MachWas));
```

<https://docs.microsoft.com/en-us/dotnet/api/system.threading.tasks.taskcreationoptions?view=netframework-4.7.2>

Task.Run(action) equates to:

```
Task.Factory.StartNew(action, CancellationToken.None,  
TaskCreationOptions.DenyChildAttach, TaskScheduler.Default);
```

Task.Factory.StartNew(action) equates to:

```
Task.Factory.StartNew(action, CancellationToken.None, TaskCreationOptions.None,  
TaskScheduler.Current)
```

TaskScheduler.Current = When not called from within a task, [Current](#) will return the [Default](#) scheduler.

TaskScheduler.Default = Threadpool

The default task scheduler is based on the .NET Framework 4 thread pool, which provides work-stealing for load-balancing, thread injection/retirement for maximum throughput, and overall good performance. It should be sufficient for most scenarios.

You can use the [TaskScheduler.FromCurrentSynchronizationContext](#) method to specify that a task should be scheduled to run on a particular thread. This is useful in frameworks such as Windows Forms and Windows Presentation Foundation where access to user interface objects is often restricted to code that is running on the same thread on which the UI object was created.

Tasks mit Parameter

- Bei Erstellung eines Tasks können Daten mitgegeben werden
- Typ: Object
- Methode des Tasks **muss** einen object-Parameter haben
- Nicht bei Task.Run(...) möglich

```
Task t = new Task(Run, x);  
t.Start();  
  
Task t2 = Task.Factory.StartNew(Run, x);
```

```
static void Run(object o)  
{  
    if (o is int x)  
    {  
        for (int i = 0; i < x; i++)  
        {  
            Console.WriteLine($"Task: {i}/{x}");  
        }  
    }  
}
```

Tasks mit Rückgabewert

- Tasks können im Gegensatz zu Threads ein Rückgabewert haben
- Dieser kann durch `t.Result` angegriffen werden
- Problem: Blockieren des Main Threads

```
Task<int> t1 = new Task<int>(GibWasZurück);
t1.Start();

//.NET 4.0
Task<int> t2 = Task.Factory.StartNew(GibWasZurück);
//.NET 4.5
Task<int> t3 = Task.Run(new Func<int>(GibWasZurück));

Task<string> t1 = Task<string>.Factory.StartNew(() =>
{
    Thread.Sleep(3000);
    return DateTime.Now.ToLongTimeString();
});

Console.WriteLine("Ich warte gerade ...");
Console.WriteLine($"Ergebnis: {t1.Result}");
```

Auf einen Task warten

- Oftmals muss auf Tasks gewartet werden
- Problem: Blockieren des Main Threads

```
Task t = Task.Run(() =>
{
    Console.WriteLine("Sehr lange Aufgabe...");
    Thread.Sleep(5000);
    Console.WriteLine("Aufgabe erledigt");
});
t.Wait();
// Task.WaitAny(t1,t2,t3);
// Task.WaitAll(t1,t2,t3);
```

Einen Task beenden

- Feste Integration mit CancellationTokens
 - Sender & Empfänger
 - Auf dem Sender wird das „Cancel“-Signal gesendet
 - Der oder die Empfänger können somit abgebrochen werden

```
CancellationTokenSource cts = new CancellationTokenSource();

Task t = Task.Run(() =>
{
    Console.WriteLine("Sehr lange Aufgabe...");
    for (int i = 0; i < 100; i++)
    {
        if (cts.Token.IsCancellationRequested)
            break;
        // Alternative:
        // cts.Token.ThrowIfCancellationRequested();

        Console.Write('#');
        Thread.Sleep(100);
    }
    Console.WriteLine("Aufgabe erledigt");
});

Thread.Sleep(2000);
cts.Cancel();
```

Due to comments, I'm posting another answer.
Consider the following code:

```
var tokenSource = new CancellationTokenSource();
CancellationToken ct = tokenSource.Token;
tokenSource.Cancel();
var task = Task.Factory.StartNew(() => {
    // Were we already canceled?
    ct.ThrowIfCancellationRequested();
    // do some processing });
```

Even if the call `tokenSource.Cancel()` is issued before the task was actually started, you'll still allocate a worker thread from thread pool, so you'll waste some system resources.

But when you specify token argument in `Task.Factory.StartNew`, the task will be cancelled immediately, without allocating a worker thread.

Exceptions im Task

```
Task t1 = Task.Run(() => throw new ArgumentException());
Task<string> t2 = Task<string>.Factory.StartNew(() => throw new FormatException());

// So bekommt man keine Exception mit !

Console.WriteLine("---ENDE---");
Console.ReadKey();
```

Richtige Vorgehensweise:

```
try
{
    // Entweder Warten
    Task.WaitAll(t1, t2);
    // oder Result abfragen
    string ergebnis = t2.Result;
}
catch (AggregateException ex)
{
    // Alle Exceptions: ex.InnerExceptions
}
```

Tasks verketten

- Tasks können mittels ContinueWith verkettet werden
- Verkettung kann auch mit Bedingungen verknüpft werden
- Auf den vorherigen Task kann in den Folgetasks zugegriffen werden
- Umgehung der Blockade des Main Threads

```
Task<double> berechne = new Task<double>(() =>
{
    Thread.Sleep(1000);
    return Math.Pow(4, 23);
});
berechne.ContinueWith(vorherigerTask => Console.WriteLine(vorherigerTask.Result));
berechne.ContinueWith(vorherigerTask => Console.WriteLine("Fehler"), TaskContinuationOptions.OnlyOnFaulted);
berechne.ContinueWith(vorherigerTask => Console.WriteLine("Erfolg"), TaskContinuationOptions.OnlyOnRanToCompletion);
berechne.Start();
```